

Model 2

```
library('rjags')

## Loading required package: coda
## Linked to JAGS 4.3.2
## Loaded modules: basemod,bugs

library(dplyr)

##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(tibble)
library(stringr)
library(ggplot2)
library(tidyr)
```

Preprocess

Load data Loading data in as a wide data frame

With ~ rows (regions) x columns (weeks)

1st column is the region names

```
uecDf = read.csv('../data/wide_weekly_scaledPer10k.csv')
regions = uecDf$Region # saving region names
n.region = length(uecDf$Region) # number of regions
uecDf = uecDf %>% select(-Region) # remove region col
n.weeks = length(names(uecDf))
head(uecDf)
```

```
##   X2023.01.01 X2023.01.08 X2023.01.15 X2023.01.22 X2023.01.29 X2023.02.05
## 1   0.9001159   6.857584   5.521329   5.289341   4.222193   4.491300
## 2   0.6823454   4.953323   3.285367   2.426117   2.021764   1.726924
## 3   0.6590512   6.013842   4.520679   3.604186   3.738056   3.233470
## 4   1.1620616   6.923950   3.776700   4.599827   3.921958   5.326116
## 5   1.0666825   7.142722   4.982352   5.022859   5.914012   5.171385
## 6   1.2769005  10.531138   6.002749   7.937845   6.489814   6.081732
##   X2023.02.12 X2023.02.19 X2023.02.26 X2023.03.05 X2023.03.12 X2023.03.19
## 1   5.252223   5.632684   6.170898   5.577007   7.599948   6.412166
## 2   1.760620   3.479119   3.462271   2.485085   2.089156   2.156549
## 3   4.325023   4.170558   3.604186   4.252940   4.345619   3.923414
```

## 4	6.439758	7.287095	7.819706	7.577610	7.432352	7.359723
## 5	3.807651	5.900510	5.860003	5.009357	7.412768	6.170556
## 6	5.699978	7.793042	7.977337	6.410830	7.635075	8.688189
##	X2023.03.26	X2023.04.02	X2023.04.09	X2023.04.16	X2023.04.23	X2023.04.30
## 1	6.996777	6.653434	4.546977	5.215105	5.725479	5.753318
## 2	2.746230	2.897862	2.232365	1.389963	1.920676	1.819588
## 3	4.613358	3.799842	2.018344	3.212875	4.283833	4.716335
## 4	8.715462	8.328108	8.037593	6.996579	8.110222	8.400737
## 5	6.373090	7.048206	5.846500	6.211063	6.589127	6.008528
## 6	9.543844	8.806664	9.267401	8.477566	8.082648	7.319141
##	X2023.05.07	X2023.05.14	X2023.05.21	X2023.05.28	X2023.06.04	X2023.06.11
## 1	6.736950	7.405077	4.825364	4.639773	4.194355	3.2385613
## 2	2.661990	2.678838	2.325029	2.737806	1.710076	0.5307131
## 3	5.272410	4.386810	2.924540	3.367340	1.966856	2.1522141
## 4	8.884929	7.504981	7.311304	8.763881	7.747077	7.9407542
## 5	5.887007	6.332584	6.062537	5.198389	3.564610	2.4574205
## 6	8.819828	8.872484	7.740386	7.121682	5.805290	6.7662561
##	X2023.06.18	X2023.06.25	X2023.07.02	X2023.07.09	X2023.07.16	X2023.07.23
## 1	4.482020	4.6212136	5.1872659	5.1315886	4.6768909	2.459080
## 2	0.463321	0.9182179	0.4212009	0.2695686	0.8087057	0.943490
## 3	2.728884	2.1522141	1.9874513	2.7288838	2.3375722	2.265488
## 4	7.529191	8.1344312	9.5385889	8.8849293	8.5459946	8.473366
## 5	3.888665	3.7806469	2.7004620	2.1198627	3.5376053	3.821154
## 6	7.095354	7.9115174	5.8184537	4.3177666	5.8842733	5.884273
##	X2023.07.30	X2023.08.06	X2023.08.13	X2023.08.20	X2023.08.27	X2023.09.03
## 1	2.7189068	2.7467454	2.820982	4.194355	3.591184	3.841732
## 2	0.7750096	0.7076175	2.004916	2.013340	1.718500	1.297299
## 3	3.1098978	2.9863257	2.605312	2.574419	2.687693	2.203702
## 4	9.1754447	7.9407542	7.771287	7.093418	7.553400	8.521785
## 5	4.5097716	4.3612462	2.497927	3.834656	5.846500	6.926685
## 6	6.2002075	6.9505510	7.845698	6.687273	6.331847	6.766256
##	X2023.09.10	X2023.09.17	X2023.09.24	X2023.10.01	X2023.10.08	X2023.10.15
## 1	4.695450	4.584095	4.435623	5.632684	5.512050	5.706920
## 2	1.575291	1.389963	1.861708	2.333453	2.316605	2.569325
## 3	2.286084	2.450847	3.593889	2.873051	2.842158	3.130493
## 4	9.804895	9.151235	9.320702	10.143829	10.095410	8.739672
## 5	5.954519	4.563781	5.319910	6.616132	5.535947	5.468436
## 6	7.924681	6.239699	5.410372	5.489356	6.134388	6.015913
##	X2023.10.22	X2023.10.29	X2023.11.05	X2023.11.12	X2023.11.19	X2023.11.26
## 1	4.491300	5.066632	6.013145	6.690552	5.911070	4.546977
## 2	2.139701	2.451389	2.855742	2.923134	3.041070	2.813622
## 3	2.924540	3.315851	3.089302	3.913116	3.738056	3.460019
## 4	9.441750	8.013383	6.923950	8.231270	9.659637	8.836510
## 5	4.860832	5.333413	5.981523	5.887007	6.251570	5.063366
## 6	5.397208	5.581503	5.805290	6.621453	7.253321	5.015454
##	X2023.12.03	X2023.12.10	X2023.12.17	X2023.12.24	X2023.12.31	X2024.01.07
## 1	4.407784	6.634875	6.653434	2.5611545	1.0857068	4.083000
## 2	2.392421	3.133735	2.771502	0.9856101	0.2021764	2.089156
## 3	4.057284	3.511507	3.892521	2.6156094	0.9061954	1.637330
## 4	8.231270	9.514379	10.095410	5.7618887	2.7841059	7.069208
## 5	6.062537	4.928343	6.602630	3.2540568	1.9713373	6.859174
## 6	5.120766	8.043157	8.990959	4.9891266	2.0535719	7.529764
##	X2024.01.14	X2024.01.21	X2024.01.28	X2024.02.04	X2024.02.11	X2024.02.18
## 1	3.953086	4.435623	5.140868	3.943807	4.714009	3.823173

## 2	3.083191	3.108463	2.939982	2.737806	3.378031	2.569325
## 3	2.677395	3.099600	2.924540	3.305554	4.294130	4.129368
## 4	7.698658	8.667043	10.192249	10.821699	12.129018	10.095410
## 5	6.481109	7.493782	7.439773	4.725809	4.253228	3.888665
## 6	6.318683	5.436700	6.805748	6.147552	6.726764	7.714059
##	X2024.02.25	X2024.03.03	X2024.03.10	X2024.03.17	X2024.03.24	X2024.03.31
## 1	4.584095	5.038793	4.751127	4.871761	3.999484	4.482020
## 2	3.285367	2.400845	2.333453	3.066343	2.712534	2.249213
## 3	2.811265	2.965730	3.109898	3.573293	2.780372	3.264363
## 4	9.369122	10.192249	9.635427	8.909139	10.168039	10.264877
## 5	3.402582	4.671799	3.605117	4.685302	5.076869	3.645624
## 6	6.239699	4.844323	4.146636	4.225619	5.568339	4.844323
##	X2024.04.07	X2024.04.14	X2024.04.21	X2024.04.28	X2024.05.05	X2024.05.12
## 1	4.751127	5.066632	4.537698	3.405593	3.507668	3.025132
## 2	2.164973	2.855742	2.948406	2.889438	2.181821	2.544053
## 3	3.923414	3.387935	3.696865	3.315851	2.831861	2.471442
## 4	8.763881	8.497575	9.732266	10.071200	7.892335	7.626029
## 5	5.306408	5.400924	3.281061	3.780647	5.076869	2.389909
## 6	6.739928	8.517058	6.042240	3.988668	3.909685	5.318225
##	X2024.05.19	X2024.05.26	X2024.06.02	X2024.06.09	X2024.06.16	X2024.06.23
## 1	4.342827	3.711818	4.500580	4.268591	3.841732	3.331357
## 2	2.965254	3.386455	2.080732	2.341877	2.822046	2.763078
## 3	2.121321	2.172809	1.657926	2.234595	2.347870	2.430251
## 4	10.434345	10.579602	10.216458	7.553400	7.335514	6.827112
## 5	2.551937	4.172214	4.037191	5.279403	4.887836	4.050693
## 6	6.516142	3.646407	4.883815	4.989127	7.582419	6.279191
##	X2024.06.30	X2024.07.07	X2024.07.14	X2024.07.21	X2024.07.28	X2024.08.04
## 1	3.071530	2.310607	2.394123	3.080809	3.544786	3.201443
## 2	2.973678	2.392421	1.432083	1.667956	1.710076	1.769044
## 3	2.780372	2.162512	2.502335	1.760902	1.688819	1.894772
## 4	8.788091	9.102816	11.233262	9.417541	7.747077	7.698658
## 5	3.348573	3.510601	4.523274	3.861661	4.266730	2.160370
## 6	6.055404	4.475734	5.699978	6.094896	5.910601	5.713142
##	X2024.08.11	X2024.08.18	X2024.08.25	X2024.09.01	X2024.09.08	X2024.09.15
## 1	3.108648	3.043691	2.431241	3.303518	2.718907	1.967264
## 2	1.819588	1.255179	1.802740	2.097580	2.392421	2.308181
## 3	2.327275	1.935963	2.584716	2.667098	2.718586	3.490912
## 4	9.030187	3.921958	0.000000	2.493591	4.793504	6.028195
## 5	3.186545	3.065024	2.349402	2.551937	2.713964	4.766316
## 6	5.555175	5.054946	4.673192	4.857487	4.752176	5.397208
##	X2024.09.22	X2024.09.29	X2024.10.06	X2024.10.13	X2024.10.20	X2024.10.27
## 1	3.238561	3.257120	3.841732	4.203634	3.182884	3.127207
## 2	2.232365	2.805198	3.344335	2.518781	2.619870	2.417693
## 3	3.326149	3.058409	3.614484	3.357042	3.398233	2.358168
## 4	6.754483	7.650239	9.417541	8.788091	8.086012	6.633435
## 5	4.712306	4.266730	5.360417	3.834656	3.051522	2.754471
## 6	6.634617	6.990043	6.739928	6.042240	5.568339	5.173422
##	X2024.11.03	X2024.11.10	X2024.11.17	X2024.11.24	X2024.12.01	X2024.12.08
## 1	2.988014	3.359195	3.758216	3.479830	4.519139	4.120118
## 2	2.754654	3.049494	2.510357	3.184279	4.009833	3.765536
## 3	2.172809	3.367340	2.780372	3.027516	4.201451	4.180856
## 4	9.030187	10.942747	9.635427	9.829104	8.715462	10.313297
## 5	2.970508	4.820325	3.186545	3.389080	5.009357	4.307237
## 6	4.673192	6.239699	6.502978	6.108060	5.739470	5.502520

##	X2024.12.15	X2024.12.22	X2024.12.29	X2025.01.05	X2025.01.12	X2025.01.19
## 1	4.816084	4.639773	1.503286	3.925248	5.428534	4.426343
## 2	3.597056	2.965254	1.120394	1.794316	3.217975	3.201127
## 3	3.810140	2.193405	1.019470	3.490912	3.470316	3.655675
## 4	8.763881	10.192249	4.987181	10.579602	11.765874	11.160633
## 5	3.537605	4.833827	1.741798	5.292906	7.264243	7.021201
## 6	5.660487	5.278733	2.566965	7.174338	8.003665	8.227452
##	X2025.01.26	X2025.02.02	X2025.02.09	X2025.02.16	X2025.02.23	X2025.03.02
## 1	3.544786	4.361386	3.971645	4.444902	4.482020	4.500580
## 2	3.504391	4.085649	4.523698	3.807656	3.748688	2.577749
## 3	5.045861	3.851330	4.242642	2.512633	3.521805	2.625907
## 4	8.521785	9.514379	9.223864	8.449156	10.385925	9.974362
## 5	6.494611	6.332584	4.914841	4.212721	4.509772	4.725809
## 6	8.477566	9.635991	9.754467	7.213829	7.450780	8.477566
##	X2025.03.09	X2025.03.16	X2025.03.23	X2025.03.30	X2025.04.06	X2025.04.13
## 1	3.600464	3.117927	2.635391	3.368475	2.440520	3.340636
## 2	1.785892	1.912252	1.069850	2.603022	3.066343	1.482627
## 3	3.027516	2.306679	1.946261	1.348995	1.606437	1.657926
## 4	8.884929	7.892335	6.270291	9.320702	7.722868	7.601820
## 5	3.497098	2.092858	3.834656	4.739311	4.104702	2.308895
## 6	6.608289	6.213371	4.041324	5.844782	4.936471	4.436242
##	X2025.04.20	X2025.04.27	X2025.05.04	X2025.05.11	X2025.05.18	X2025.05.25
## 1	3.962366	3.860291	2.421961	2.394123	2.691068	3.071530
## 2	2.367149	1.272027	1.339419	1.196211	1.010882	1.516323
## 3	1.668223	1.935963	1.781498	2.008047	1.832986	2.399358
## 4	8.739672	9.611218	8.400737	8.497575	9.248074	8.521785
## 5	2.281890	2.646453	2.106360	2.673457	2.362904	2.200877
## 6	4.778504	5.515683	4.594209	5.423536	6.266027	4.660029
##	X2025.06.01	X2025.06.08	X2025.06.15	X2025.06.22	X2025.06.29	X2025.07.06
## 1	2.783864	1.679598	2.310607	3.683980	2.728186	2.635391
## 2	1.828012	1.945948	1.760620	1.566867	1.263603	1.592139
## 3	2.121321	2.275786	1.585842	1.802093	1.853581	1.740307
## 4	8.473366	8.134431	8.667043	6.585016	9.659637	8.933349
## 5	1.579770	2.106360	1.755300	2.470923	3.429587	2.970508
## 6	5.792126	6.121224	6.423994	5.910601	6.410830	5.607831
##	X2025.07.13	X2025.07.20	X2025.07.27	X2025.08.03	X2025.08.10	X2025.08.17
## 1	1.744555	1.985823	1.818791	2.022941	1.865189	2.524036
## 2	1.499475	1.524747	1.693228	0.951914	1.330995	1.853284
## 3	2.069833	2.471442	2.162512	1.905070	1.997749	2.337572
## 4	6.657645	7.795497	8.594414	9.441750	6.778693	5.568212
## 5	3.335071	2.470923	3.254057	3.159541	2.308895	2.943504
## 6	4.594209	7.069026	7.134846	6.937387	6.094896	7.740386
##	X2025.08.24	X2025.08.31	X2025.09.07	X2025.09.14	X2025.09.21	X2025.09.28
## 1	1.429050	2.570434	2.533316	1.800232	1.967264	1.707436
## 2	1.861708	1.625835	1.533171	1.743772	1.769044	1.886980
## 3	2.842158	2.564121	2.368465	3.243768	3.573293	2.419954
## 4	9.369122	9.199654	9.272283	7.964964	8.884929	9.320702
## 5	2.700462	3.591615	4.644795	3.132536	3.146038	4.253228
## 6	8.240615	7.951009	7.134846	8.582877	7.253321	4.080816
##	X2025.10.05	X2025.10.12	X2025.10.19	X2025.10.26	X2025.11.02	X2025.11.09
## 1	2.421961	2.468359	1.735275	1.716716	2.273489	2.746745
## 2	2.375573	1.642683	1.246755	1.280451	1.457355	2.063884
## 3	2.131619	2.770075	2.955433	2.862754	2.728884	4.458893
## 4	10.700651	6.972370	4.502989	3.728281	4.260893	4.236683

```
## 5    4.536776    5.211892    5.670970    4.320739    5.076869    6.197560
## 6    5.291897    6.884731    4.778504    5.568339    4.804832    6.476650
##    X2025.11.16
## 1    2.895218
## 2    1.996492
## 3    3.665972
## 4    4.067216
## 5    2.808481
## 6    5.620995
```

Reformat Dataframe to Matrix Formatting as matrix with ~ rows (regions) x columns (weeks)

```
uecMat = as.matrix(uecDf)
```

Model 2 Specification

```
output_folder = "../data/models/v2/"
save_output   = TRUE
if (save_output) dir.create(output_folder, recursive = TRUE, showWarnings = FALSE)

model=
"
model{
  # Iterate through the regions
  for(i in 1:I){
    #-----
    #Likelihood
    #-----
    # Set first data point in region i to normal
    y[i,1] ~ dnorm(mu[i,1], tau[i])
    # Set second on data points to the AR1 model with a mean with a seasonal component
    for(t in 2:T){
      y[i,t] ~ dnorm(mu[i,t] +
        (phi * (y[i,t-1] - mu[i,t-1])), tau[i])
    }
    # Assign mean to each time point
    # Beta ~ cosine coeffecient scaler
    # Gamma ~ sine coeffecient scaler
    for(t in 1:T){
      mu[i,t] <- alpha[i] +
        beta[i]      * cos((2 * pi) * (t/52)) +
        gamma[i]     * sin((2 * pi) * (t/52)) +

        delta_pre   * ny_pre[t]  +
        delta_mid   * ny_mid[t]  +
        delta_post  * ny_post[t] +

        sigma_pre   * fr_pre[t]  * mw[i] +
        sigma_mid   * fr_mid[t]  * mw[i] +
        sigma_post  * fr_post[t] * mw[i]
    }
    #Full conditional mean used in the likelihood
    fullmod[i,1] <- mu[i,1]
    for(t in 2:T){
```

```

    fullmod[i,t] <- mu[i,t] + phi * (y[i,t-1] - mu[i,t-1])
  }
  #Residuals
  for(t in 1:T){
    resid[i,t] <- y[i,t] - fullmod[i,t]
  }
  #Region specific priors
  alpha[i] ~ dnorm(0, 0.001)
  beta[i] ~ dnorm(0, 0.001)
  gamma[i] ~ dnorm(0, 0.001)
  tau[i] ~ dgamma(0.001, 0.001)
}
#non region specific priorss
phi ~ dunif(-1, 1)
sigma_pre ~ dnorm(0, 0.001)
sigma_mid ~ dnorm(0, 0.001)
sigma_post ~ dnorm(0, 0.001)
delta_pre ~ dnorm(0, 0.001)
delta_mid ~ dnorm(0, 0.001)
delta_post ~ dnorm(0, 0.001)
}
"
# New year indicators: pre, mid, post
week_mod = (1:n.weeks) %% 52
# as numeric makes a mask
ny_pre = as.numeric(week_mod == 0) # week before dip
ny_mid = as.numeric(week_mod == 1) # the dip
ny_post = as.numeric(week_mod == 2) # week after dip
fr_pre = as.numeric((1:n.weeks) == 86) # week before reset
fr_mid = as.numeric((1:n.weeks) == 87) # the reset
fr_post = as.numeric((1:n.weeks) == 88) # week after reset
mw = as.numeric(regions == "HSE Mid West") # 1 only for Mid West
# fit model
exp_jags = jags.model(textConnection(model),
                      list(y = uecMat,
                           I = n.region,
                           T = n.weeks,
                           pi = pi,
                           ny_pre = ny_pre,
                           ny_mid = ny_mid,
                           ny_post = ny_post,
                           fr_pre = fr_pre,
                           fr_mid = fr_mid,
                           fr_post = fr_post,
                           mw = mw), n.chains = 4)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 906
##   Unobserved stochastic nodes: 31
##   Total graph size: 6883
##

```

```
## Initializing model
update(exp_jags, n.iter = 10000)
# Parameters to watch
model_parameters = c("alpha", "beta", "gamma",
                     "tau", "phi", "mu", "fullmod",
                     "delta_pre", "delta_mid", "delta_post",
                     "sigma_pre", "sigma_mid", "sigma_post", "resid")

# Sample
exp_sim=coda.samples(model=exp_jags,
                     variable.names=model_parameters,
                     n.iter=20000)

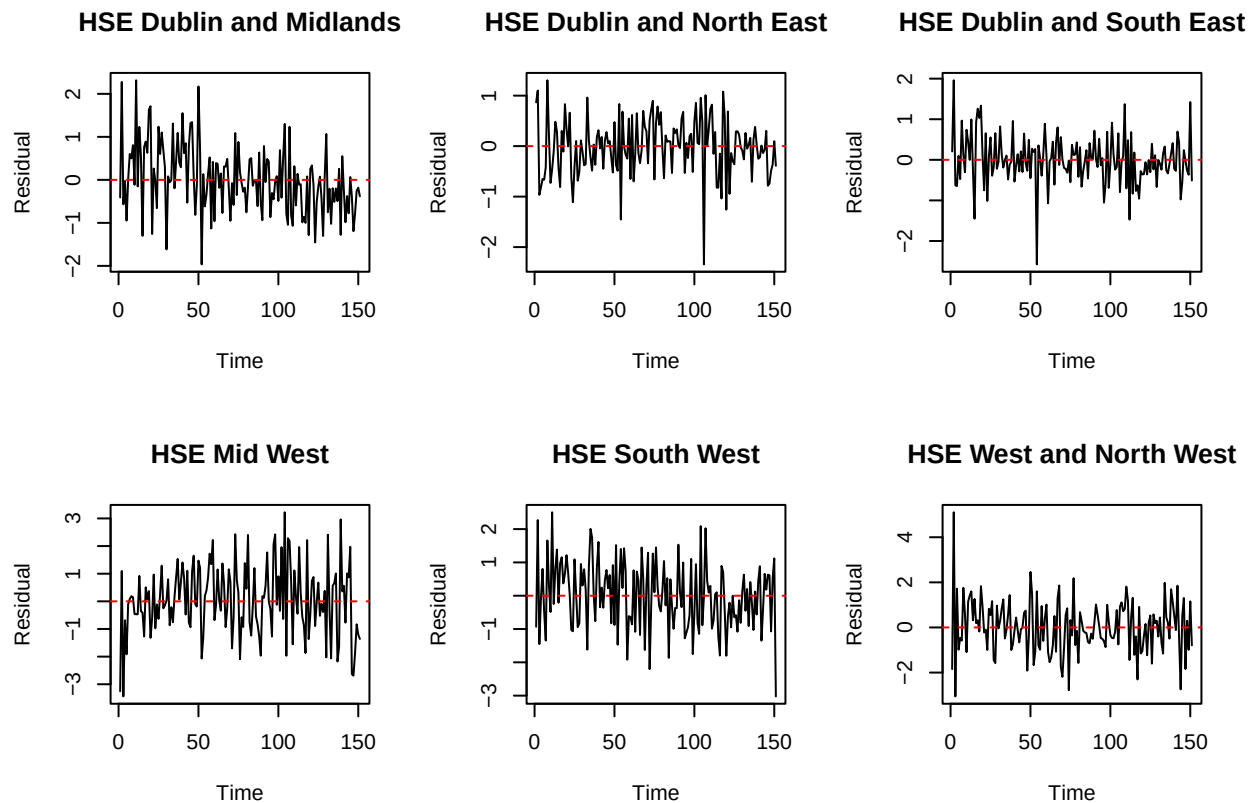
# Extract values to something usable
jags_sum <- summary(exp_sim)
df.stats = cbind(as.data.frame(jags_sum$statistics), as.data.frame(jags_sum$quantiles)) %>%
  rownames_to_column(var = "index")
```

```
fullmod_rows = df.stats %>%
  filter(grepl("fullmod", index)) %>%
  mutate(
    i = as.integer(str_extract(index, "(?<=\\[\\]\\d+")),
    t = as.integer(str_extract(index, "\\d+(?=\\[\\])"))
  ) %>%
  arrange(i, t)
matrix_fullmod = matrix(fullmod_rows$Mean, nrow = n.region, ncol = n.weeks, byrow = TRUE)
df_fitted = as.data.frame(t(matrix_fullmod))
colnames(df_fitted) = regions
df_fitted$time = seq_len(n.weeks)
df_fitted = df_fitted %>% pivot_longer(-time, names_to = "Region", values_to = "fitted")
if (save_output) write.csv(df_fitted, paste0(output_folder, "fitted.csv"), row.names = FALSE)
```

Save fitted values

Check Residuals

```
# residuals = observed - fitted
resid_vector = as.vector(df.stats %>% filter(grepl("resid", index)) %>% pull(Mean))
matrix_resid = matrix(resid_vector, nrow = length(resid_vector)/6, ncol = 6, byrow = T)
df_resid = as.data.frame(matrix_resid)
colnames(df_resid) = regions
df_resid$time = seq_len(nrow(df_resid))
# 2x3 residual plot (one per region)
par(mfrow = c(2, 3))
for (j in 1:6) {
  plot(df_resid$time, matrix_resid[, j], type = "l",
       main = regions[j], xlab = "Time", ylab = "Residual")
  abline(h = 0, col = "red", lty = 2)
}
```

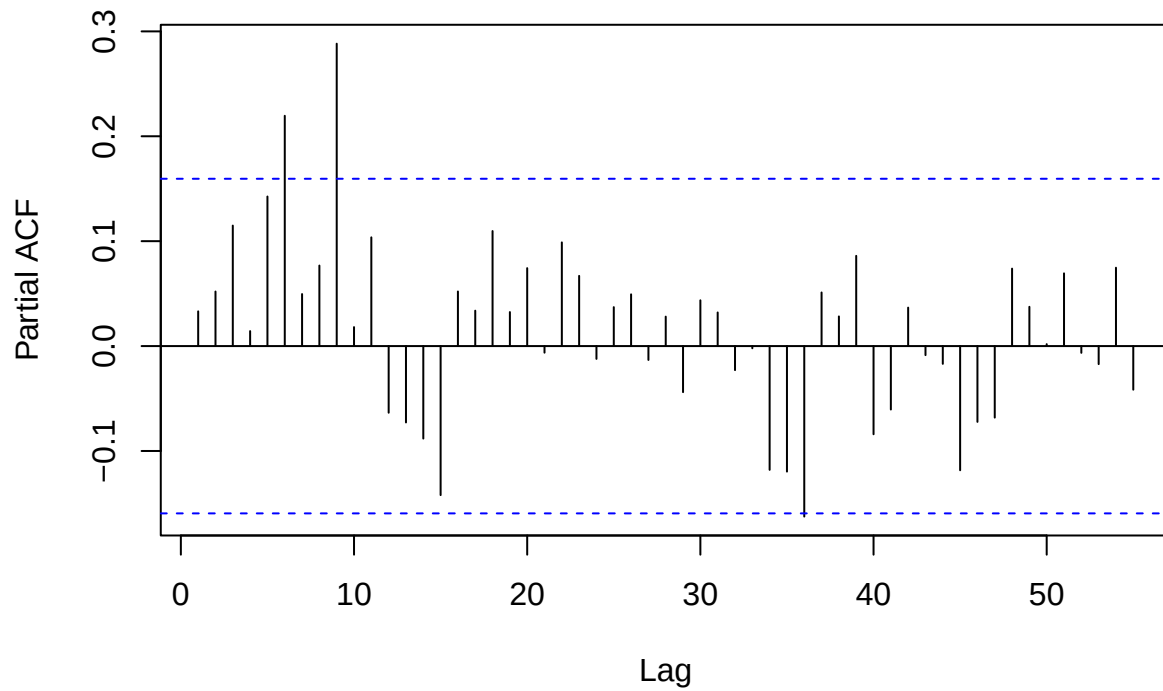


```
par(mfrow = c(1, 1))
# resid outlier df
df_outliers = df_resid %>%
  pivot_longer(-time, names_to="Region", values_to="Residual") %>%
  filter(abs(Residual) > 3)
df_outliers
```

```
## # A tibble: 6 x 3
##   time Region          Residual
##   <int> <chr>          <dbl>
## 1     1 1 HSE Mid West      -3.25
## 2     2 2 HSE West and North West  5.11
## 3     3 3 HSE Mid West      -3.44
## 4     3 3 HSE West and North West  -3.06
## 5   104 5 HSE Mid West       3.22
## 6   151 6 HSE South West     -3.02
```

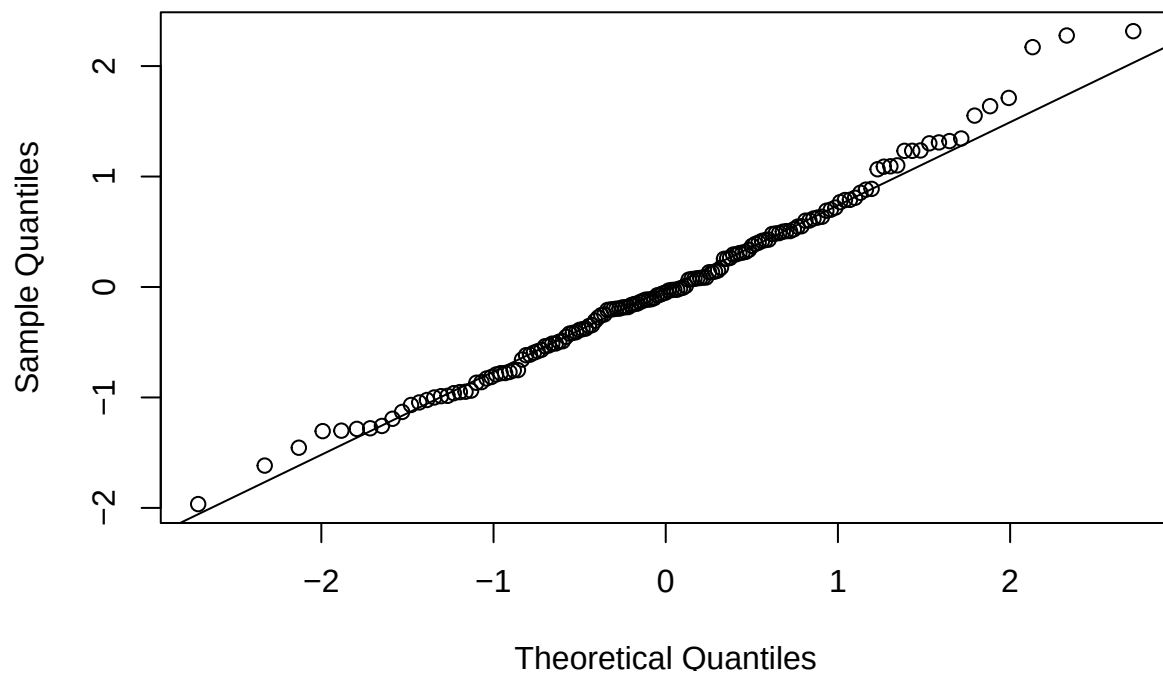
```
# acf
pacf(matrix_resid[,1], lag.max=55, main="ACF of residuals") # check remaining autocorrelation
```

ACF of residuals



```
# qq  
qqnorm(matrix_resid[,1]); qqline(matrix_resid[,1]) # normality check
```

Normal Q-Q Plot



```
# gelman stat check  
#gelman.diag(exp_sim, multivariate = FALSE)
```

```
dic_v2 = dic.samples(exp_jags, n.iter = 20000)
dic_v2
```

DIC

```
## Mean deviance: 2319
## penalty 31.82
## Penalized deviance: 2350
```

```
df_dic = data.frame(
  deviance = sum(dic_v2$deviance),
  penalty = sum(dic_v2$penalty),
  DIC = sum(dic_v2$deviance) + sum(dic_v2$penalty)
)
if (save_output) write.csv(df_dic, paste0(output_folder, "dic.csv"), row.names = FALSE)
```

```
cat("Pre NY weeks:", which(ny_pre == 1), "\n")
```

```
## Pre NY weeks: 52 104
```

```
cat("Mid NY weeks:", which(ny_mid == 1), "\n")
```

```
## Mid NY weeks: 1 53 105
```

```
cat("Post NY weeks:", which(ny_post == 1), "\n")
```

```
## Post NY weeks: 2 54 106
```

```
cat("Pre fr weeks:", which(fr_pre == 1), "\n")
```

```
## Pre fr weeks: 86
```

```
cat("Mid fr weeks:", which(fr_mid == 1), "\n")
```

```
## Mid fr weeks: 87
```

```
cat("Post fr weeks:", which(fr_post == 1), "\n")
```

```
## Post fr weeks: 88
```

Check Convergence

Trace plots and posterior densities for all scalar parameters

```
#params = c("alpha[1]", "beta[1]", "gamma[1]", "tau[1]", "delta[1]", "sigma[1]", "phi")
#for (p in params) {
#  par(mfrow = c(1, 2), mar = c(2, 4, 2, 1))
#  traceplot(exp_sim[, p], main = paste("Trace:", p))
#  densplot(exp_sim[, p], main = paste("Density:", p))
#}
#par(mfrow = c(1, 1))
```

Gelman Rubin Statistic All are 1.

```
# fit model
exp_jags = jags.model(textConnection(model),
  list(y = uecMat,
    I = n.region,
    T = n.weeks,
```

```

        pi = pi,
        ny_pre = ny_pre,
        ny_mid = ny_mid,
        ny_post = ny_post,
        fr_pre = fr_pre,
        fr_mid = fr_mid,
        fr_post = fr_post,
        mw = mw), n.chains = 4)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 906
##   Unobserved stochastic nodes: 31
##   Total graph size: 6883
##
## Initializing model

update(exp_jags, n.iter = 10000)
# Parameters to watch
model_parameters = c("alpha", "beta", "gamma",
                     "tau", "phi",
                     "delta_pre", "delta_mid", "delta_post",
                     "sigma_pre", "sigma_mid", "sigma_post")

# Sample
exp_sim=coda.samples(model=exp_jags,
                     variable.names=model_parameters,
                     n.iter=20000)

gelman_results = gelman.diag(exp_sim, multivariate = FALSE)
gelman_results

## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## alpha[1]           1           1
## alpha[2]           1           1
## alpha[3]           1           1
## alpha[4]           1           1
## alpha[5]           1           1
## alpha[6]           1           1
## beta[1]            1           1
## beta[2]            1           1
## beta[3]            1           1
## beta[4]            1           1
## beta[5]            1           1
## beta[6]            1           1
## delta_mid          1           1
## delta_post         1           1
## delta_pre          1           1
## gamma[1]           1           1
## gamma[2]           1           1
## gamma[3]           1           1

```

```
## gamma[4]          1          1
## gamma[5]          1          1
## gamma[6]          1          1
## phi               1          1
## sigma_mid         1          1
## sigma_post        1          1
## sigma_pre         1          1
## tau[1]            1          1
## tau[2]            1          1
## tau[3]            1          1
## tau[4]            1          1
## tau[5]            1          1
## tau[6]            1          1
```

```
if (save_output) {
  samples_with_chain = do.call(rbind, lapply(seq_along(exp_sim), function(ch) {
    df = as.data.frame(exp_sim[[ch]])
    df$chain = ch
    df
  }))
  write.csv(samples_with_chain,
            paste0(output_folder, "raw_samples.csv"),
            row.names = FALSE)
}
```

Save